

RemSolution — Project Overview

Malek MSELLEM

July 3, 2026

CONTENTS

RemSolution — Full Project Description	
1. What it is	
2. Architecture	
3. Technology stack	
Backend	
Application patterns / libraries	
Frontend	
Build / DevOps	
Testing (scaffolding present)	
4. Cross-cutting techniques	
5. Domain model (14 entities)	
6. What is actually done (feature matrix)	
7. What is partial / not done	
8. Recent changes (applied)	
9. Summary	

RemSolution — Full Project Description

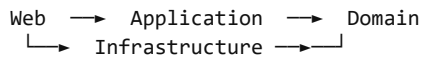
1. What it is

RemSolution is a **car-rental management system** (backend API + Angular SPA). It is built on the **Ardalis / Jason Taylor “Clean Architecture” solution template** for .NET, with the template’s sample content (Todo / WeatherForecast) being stripped out and replaced by a real rental domain: brands, car models, cars, clients, rentings, reservations, payments, expenses, and extra services.

The domain vocabulary and field names (Matricule, CIN, Passeport, Deliverance) indicate a **French / North-African business context** (consistent with the `hrmaps.fr` owner and the French build output).

2. Architecture

Classic **Clean Architecture** — four projects, with dependencies pointing inward:



Layer	Project	Responsibility
Domain	<code>RemSolution.Domain</code>	Entities, enums, value objects, domain events, exceptions. No external dependencies.
Application	<code>RemSolution.Application</code>	Use cases (CQRS commands / queries), DTOs, validation, mapping, pipeline behaviours, interfaces.
Infrastructure	<code>RemSolution.Infrastructure</code>	EF Core <code>DbContext</code> , entity configurations, migrations, Identity, interceptors, external services.
Web	<code>RemSolution.Web</code>	Minimal-API endpoints, host wiring, Swagger, Angular SPA host.

The **Web layer is the composition root**: `Program.cs` calls `AddApplicationServices()`, `AddInfrastructureServices()`, and `AddWebServices()`.

3. Technology stack

BACKEND

- .NET 9** (`Directory.Build.props` targets `net9.0`), building with **SDK 10.0.101** (`global.json`, roll-forward) — *a version mismatch worth reconciling*.

- **ASP.NET Core Minimal APIs** — endpoints grouped via a custom `EndpointGroupBase` and reflection (`MapEndpoints`).
- **Entity Framework Core 9** on **SQL Server** (LocalDB in development).
- **ASP.NET Core Identity** for authentication (users, roles, policies).
- **NSwag** for OpenAPI / Swagger generation (served at `/api`); it also generates the Angular TypeScript client.
- **Azure Key Vault** integration (optional, via `DefaultAzureCredential`).
- **Health checks** at `/health` (includes a `DbContext` check).

APPLICATION PATTERNS / LIBRARIES

- **MediatR 12** — CQRS mediator.
- **FluentValidation 11** — request validation.
- **Mapster 10** — object mapping (**migrated from AutoMapper** to remove a CVE and a licensing concern).
- **Ardalis.GuardClauses** — guard helpers.

FRONTEND

- **Angular 18** (module-based, not standalone) with **TypeScript 5.4**.
- **Angular Material 18** + **ngx-bootstrap** + **Bootstrap 5** (two UI systems present).
- **RxJS**, plus the auto-generated **NSwag TypeScript API client** (`web-api-client.ts`).

BUILD / DEVOPS

- **Central Package Management** (`Directory.Packages.props`).
- **Cake** build script (`build.cake`), **Azure Developer CLI** (`azure.yaml` + Bicep `infra/`), **.devcontainer**, and **GitHub** workflows.

TESTING (SCAFFOLDING PRESENT)

- `Domain.UnitTests`, `Application.UnitTests` (NUnit + FluentAssertions + Moq), `Application.FunctionalTests` (Testcontainers / Respawn), `Infrastructure.IntegrationTests`, `Web.AcceptanceTests` (SpecFlow + Playwright).

4. Cross-cutting techniques

- **MediatR pipeline behaviours:** `UnhandledException` → `Authorization` → `Validation` → `Performance` → `Logging`.
- **EF Core interceptors:**
 - `AuditableEntityInterceptor` — auto-sets `Created` / `LastModified` and by-whom on `BaseAuditableEntity`.
 - `DispatchDomainEventsInterceptor` — dispatches domain events on `SaveChanges`.

- **Domain events** via `BaseEntity.AddDomainEvent(...)`.
 - **CQRS vertical slices** — each feature folder has Commands / Queries / DTOs / EventHandlers.
 - **Mapster projection** — `ProjectToType<T>()` produces efficient SQL projections (no over-fetching).
 - **Global** `RequireAuthorization()` on all endpoint groups; policy-based authorization (`CanPurge` → Administrator).
 - **Custom exception handling** (`CustomExceptionHandler`) → RFC-7807 problem details.
-

5. Domain model (14 entities)

All entities inherit `BaseAuditableEntity` (Id + audit fields).

- **Brand** → has many **ModelCar** → has many **Car**
- **Car** — matricule, first-circulation date, color, power, `FuelType` (Gasoline / Diesel), image; has Expenses and Rentings
- **Client** — rich identity record: name, birth info, **CIN**, **Passeport**, **Driving Licence** (each with number / date / place / country + image), multiple `Country` foreign keys
- **Country** — reference lookup
- **Renting** — car + primary/secondary client, dates, mileage, price, `RentingState` (Done / InProgress / NotYet); has ExtraServices, RentingHistories, Reservations
- **Reservation** — client, dates, price, payed price, links to Renting
- **RentingHistory** — snapshot / audit of renting periods
- **Payment** — client payments
- **Expense + ExpenseType** — per-car expenses (with mileage / month-based notification thresholds)
- **ExtraService + ExtraServicesType** — add-ons on a renting
- **User** — a Domain user entity that *duplicates* Identity's `ApplicationUser` (a known redundancy)

Enums: `FuelType`, `RentingState`, `PriorityLevel` (leftover).

6. What is actually done (feature matrix)

Feature	Create	Update	Delete	GetById	List	Paginated	Endpoint	Angular UI
Car	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
ModelCar	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes
Brand	Yes	No	Yes	Yes	Yes	No	Yes	Yes
Country	Yes	No	Yes	Yes	Yes	No	Yes	No
Client, Renting, Reservation, Payment, Expense(Type), ExtraService(Type), User	No	No	No	No	No	No	No	No

Infrastructure done: DbContext with all 14 entities + EF configurations, one migration (`InitialCreate` , includes every table), Identity, seeding (admin user + Administrator role), interceptors, health checks, Swagger, NSwag client generation.

Frontend done: Angular 18 shell with nav-menu, home, and CRUD components for **brand / model-car / car**; Material table / paginator wired; authorize HTTP interceptor.

Verified working: solution builds clean (0 warnings / 0 errors); all 10 unit tests pass; Mapster mappings validated.

7. What is partial / not done

- **Core business flow is missing** — Renting, Reservation, Payment, and Client have entities + DB tables but **no commands / queries / endpoints / UI**. You cannot actually rent a car yet.
- **Incomplete CRUD** — Brand / ModelCar / Country lack Update commands.
- **Duplicate user identity** — `Domain.User` vs Identity `ApplicationUser` (unresolved source of truth).
- **Domain events are template leftovers** — `CarCompletedEvent` is raised on *create*; Deleted events are defined but never raised; handlers only log.
- **Hardcoded connection string** in `Infrastructure/DependencyInjection.cs` (the configuration line is commented out).
- **Model drift** — `PendingModelChangesWarning` is suppressed and the migration still contains Todo tables; only one migration exists.
- **No tests for the new domain** — the old Todo tests were deleted; the test projects are empty scaffolding.

- **Version inconsistencies** — net9 vs SDK 10, plus stale `obj/net8.0` build artifacts.
 - **Two UI frameworks** (Material + Bootstrap) loaded together.
-

8. Recent changes (applied)

1. Added `Id` to `CarDto` / `CountryDto` / `ModelCarDto`.
 2. Fixed the wrong `NotFoundException` entity name in the Country **and** ModelCar `GetById` queries.
 3. Renamed three mis-named files (`CountryDto.cs` , `GetCountryByIdQuery.cs` , `ExpenseConfiguration.cs`).
 4. **Migrated AutoMapper → Mapster** (removed CVE-2026-32933 and the new commercial-license issue); ported DI registration, four DTO mappings, ten query handlers, the mapping helper, and the mapping test.
-

9. Summary

A well-architected **.NET 9 Clean Architecture + Angular 18** car-rental application with a solid, complete foundation and a rich data model — but only about **30% of the use cases are implemented**, and the central “renting” workflow is still just data tables without behavior.